Anabela Petrushevikj D214 MSDA Capstone

In [1]:
```
# Libraries
import os
from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from scipy.stats import normaltest, probplot
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error
```

In [2]:
```
# Loading the dataset
df = pd.read_csv('used_cars_D214.csv')
```

In [3]:
```
# Output folder
OUTPUT_DIR = Path("output")
OUTPUT_DIR.mkdir(exist_ok=True)
```

In [4]:
```
# Load fissrt 5 rows of the data
df.head()
```

Out[4]:

	brand	model	model_year	milage	fuel_type	engine	transmission	ext_col	int_col	accic
0	Ford	Utility Police Interceptor Base	2013	51,000 mi.	E85 Flex Fuel	300.0HP 3.7L V6 Cylinder Engine Flex Fuel Capa...	6-Speed A/T	Black	Black	At l acci dam repo
1	Hyundai	Palisade SEL	2021	34,742 mi.	Gasoline	3.8L V6 24V GDI DOHC	8-Speed Automatic	Moonlight Cloud	Gray	At l acci dam repo
2	Lexus	RX 350 RX 350	2022	22,372 mi.	Gasoline	3.5 Liter DOHC	Automatic	Blue	Black	N repo
3	INFINITI	Q50 Hybrid Sport	2015	88,900 mi.	Hybrid	354.0HP 3.5L V6 Cylinder Engine Gas/Electric H...	7-Speed A/T	Black	Black	N repo
4	Audi	Q3 45 S line Premium Plus	2021	9,835 mi.	Gasoline	2.0L I4 16V GDI DOHC Turbo	8-Speed Automatic	Glacier White Metallic	Black	N repo

In [5]:

```
# Shape of data
df.shape
```

Out[5]:

```
(4009, 12)
```

In [6]:

```
# STANDARDIZE COLUMN NAMES
df.columns = df.columns.str.strip().str.lower()
df.rename(columns={"milage": "mileage"}, inplace=True)

print("\nColumn names:")
print(df.columns.tolist())
```

```
Column names:
['brand', 'model', 'model_year', 'mileage', 'fuel_type', 'engine', 'transmission', 'ext_col', 'in
t_col', 'accident', 'clean_title', 'price']
```

In [7]:

```
# Data types
print("\nData types:")
print(df.dtypes)
```

```
Data types:
brand           object
model           object
model_year       int64
mileage         object
fuel_type       object
engine          object
transmission    object
ext_col         object
int_col         object
accident        object
clean_title     object
price           object
dtype: object
```

In [8]:
```python
# Check for duplicates
duplicate_count = df.duplicated().sum()
print("\nDuplicate rows found:", duplicate_count)
```

```
Duplicate rows found: 0
```

In [9]:
```python
# Check for missing values
missing_values = df.isnull().sum()
missing_percent = (df.isnull().sum() / len(df)) * 100

missing_summary = pd.DataFrame({
    "Missing Values": missing_values,
    "Percent Missing": missing_percent.round(2)
}).sort_values(by="Missing Values", ascending=False)

print("\nMissing Value Summary:")
print(missing_summary)

missing_summary.to_csv(OUTPUT_DIR / "missing_values_summary.csv")
```

```
Missing Value Summary:
              Missing Values  Percent Missing
clean_title              596            14.87
fuel_type                170             4.24
accident                 113             2.82
brand                      0             0.00
model                      0             0.00
model_year                 0             0.00
mileage                    0             0.00
engine                     0             0.00
transmission               0             0.00
ext_col                    0             0.00
int_col                    0             0.00
price                      0             0.00
```

In [10]:
```python
# Clean Price column
df["price"] = df["price"].astype(str).str.replace(r"[\$,]", "", regex=True)
df["price"] = pd.to_numeric(df["price"], errors="coerce")
```

In [11]:
```python
# Clean mileage column
df["mileage"] = df["mileage"].astype(str).str.replace(r"[^0-9.]", "", regex=True)
df["mileage"] = pd.to_numeric(df["mileage"], errors="coerce")
```

In [12]:
```python
# Features
current_year = 2026
```

```python
# Vehicle age
df["vehicle_age"] = current_year - df["model_year"]

# Extract horsepower from engine text if available
df["horsepower"] = df["engine"].astype(str).str.extract(r"(\d+(?:\.\d+)?)HP", expand=False)
df["horsepower"] = pd.to_numeric(df["horsepower"], errors="coerce")

# Extract engine liters from engine text if available
df["engine_liters"] = df["engine"].astype(str).str.extract(r"(\d+(?:\.\d+)?)L", expand=False)
df["engine_liters"] = pd.to_numeric(df["engine_liters"], errors="coerce")

# Simplify transmission values
def simplify_transmission(x):
    x = str(x).strip().lower()
    if "manual" in x or "m/t" in x:
        return "Manual"
    elif "cvt" in x:
        return "CVT"
    elif "automatic" in x or "a/t" in x:
        return "Automatic"
    else:
        return "Other"

df["transmission_group"] = df["transmission"].apply(simplify_transmission)

# Fill missing categories
df["fuel_type"] = df["fuel_type"].fillna("Unknown")
df["accident"] = df["accident"].fillna("Unknown")
df["clean_title"] = df["clean_title"].fillna("Unknown")
df["brand"] = df["brand"].fillna("Unknown")

# Accident flag
df["accident_flag"] = df["accident"].map({
    "None reported": 0,
    "At least 1 accident or damage reported": 1,
    "Unknown": np.nan
})

# Clean title flag
df["clean_title_flag"] = df["clean_title"].map({
    "Yes": 1,
    "Unknown": np.nan
})

# Keep top 15 brands, all others become Other
top_brands = df["brand"].value_counts().head(15).index
df["brand_group"] = df["brand"].apply(lambda x: x if x in top_brands else "Other")

print("\nShape after feature engineering:", df.shape)
```

```
Shape after feature engineering: (4009, 19)
```

In [13]:

```python
# Remove invalid rows
df = df[df["price"] > 0]
df = df[df["mileage"] >= 0]
df = df[df["vehicle_age"] >= 0]

print("\nShape after removing invalid rows:", df.shape)
```

```
Shape after removing invalid rows: (4009, 19)
```

In [14]:
```python
# Check for outliers using IQR
numeric_cols = ["price", "mileage", "horsepower", "engine_liters"]

print("\nOutlier counts before removal:")
outlier_summary_list = []

for col in numeric_cols:
    q1 = df[col].quantile(0.25)
    q3 = df[col].quantile(0.75)
    iqr = q3 - q1
    lower = q1 - 1.5 * iqr
    upper = q3 + 1.5 * iqr

    outlier_count = df[(df[col] < lower) | (df[col] > upper)].shape[0]

    print(f"{col}: {outlier_count} outliers")

    outlier_summary_list.append({
        "Column": col,
        "Q1": q1,
        "Q3": q3,
        "IQR": iqr,
        "Lower Bound": lower,
        "Upper Bound": upper,
        "Outlier Count": outlier_count
    })

outlier_summary = pd.DataFrame(outlier_summary_list)
outlier_summary.to_csv(OUTPUT_DIR / "outlier_summary_before_removal.csv", index=False)
```

```
Outlier counts before removal:
price: 244 outliers
mileage: 69 outliers
horsepower: 63 outliers
engine_liters: 6 outliers
```

In [15]:
```python
# Remove outliers one column at a time using IQR
for col in numeric_cols:
    q1 = df[col].quantile(0.25)
    q3 = df[col].quantile(0.75)
    iqr = q3 - q1
    lower = q1 - 1.5 * iqr
    upper = q3 + 1.5 * iqr

    df = df[(df[col].isna()) | ((df[col] >= lower) & (df[col] <= upper))]

print("\nShape after outlier removal:", df.shape)
```

```
Shape after outlier removal: (3648, 19)
```

In [16]:
```python
# Normality test
# Original price
price_stat, price_p = normaltest(df["price"].dropna())
print(price_p)
```

```
4.1635082309595784e-76
```

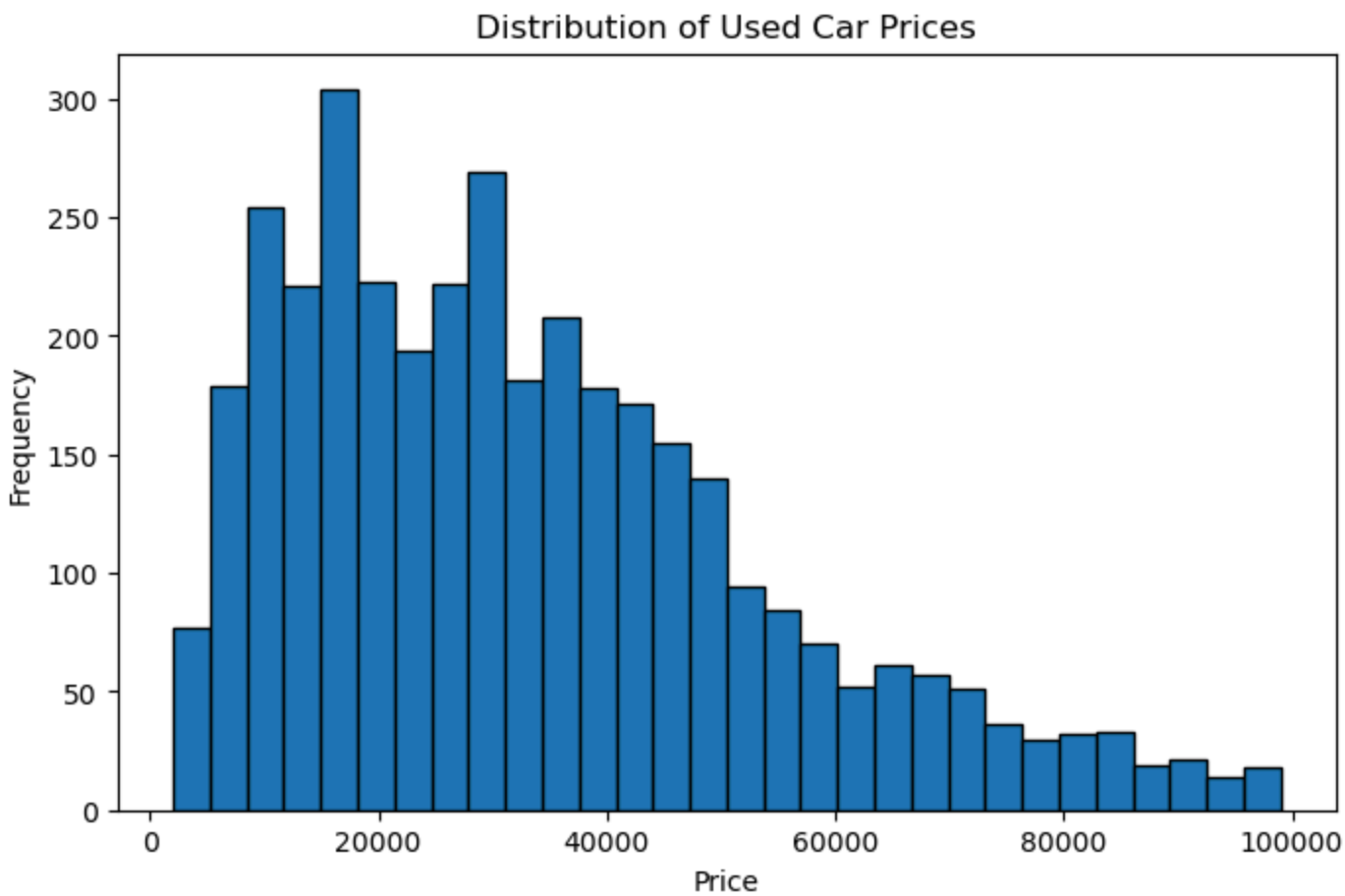
In [17]:
```python
# Log-transform price
df["log_price"] = np.log1p(df["price"])
```

```
log_price_stat, log_price_p = normaltest(df["log_price"].dropna())

normality_results = pd.DataFrame({
    "Variable": ["price", "log_price"],
    "Statistic": [price_stat, log_price_stat],
    "p_value": [price_p, log_price_p]
})

print("\nNormality Test Results:")
print(normality_results)

normality_results.to_csv(OUTPUT_DIR / "normality_results.csv", index=False)
```

```
Normality Test Results:
    Variable   Statistic       p_value
0      price  347.140218  4.163508e-76
1  log_price  167.399177  4.464065e-37
```

In [18]:

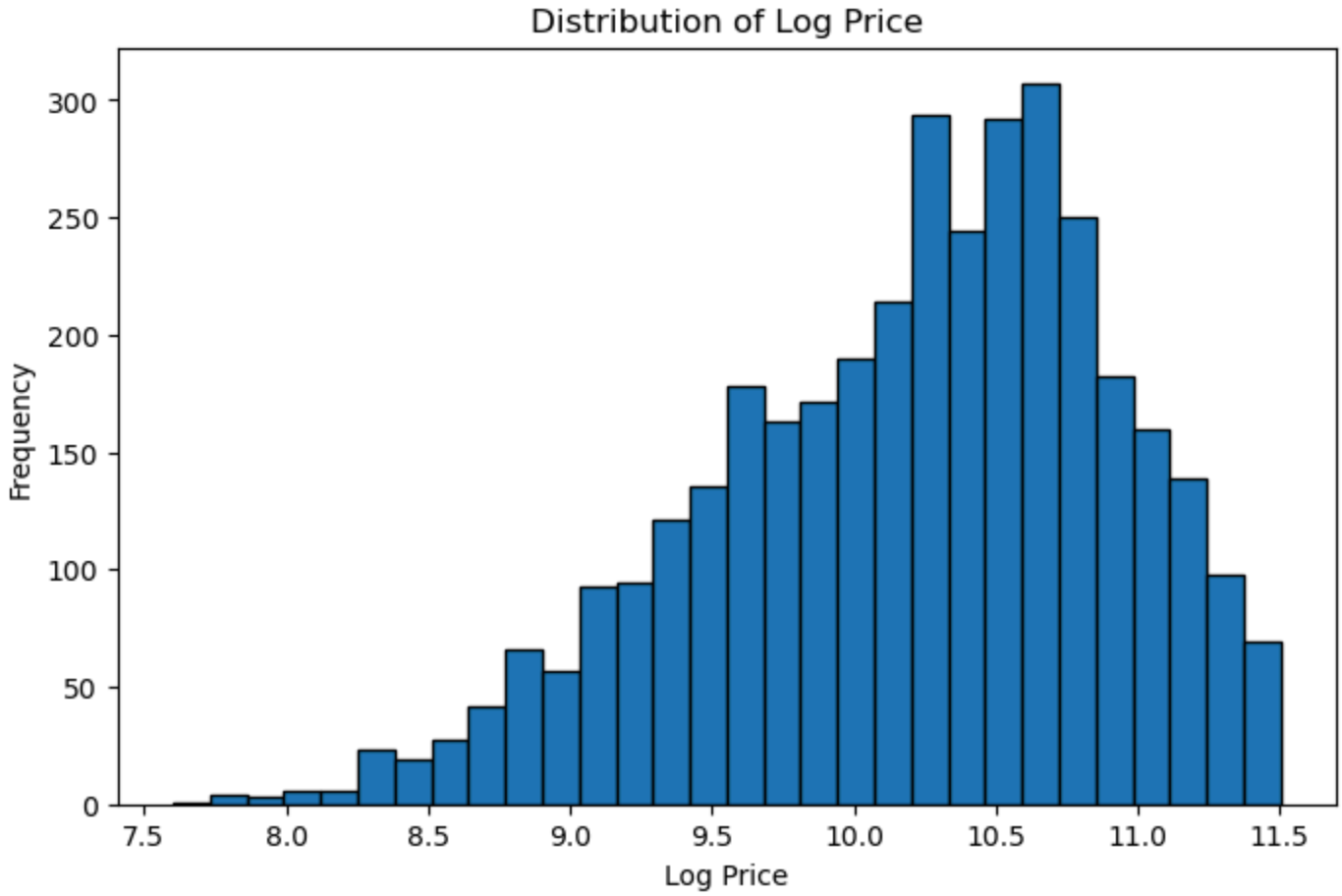
```
# Histogram: Price
plt.figure(figsize=(8, 5))
plt.hist(df["price"].dropna(), bins=30, edgecolor="black")
plt.title("Distribution of Used Car Prices")
plt.xlabel("Price")
plt.ylabel("Frequency")
plt.show()
```



In [19]:

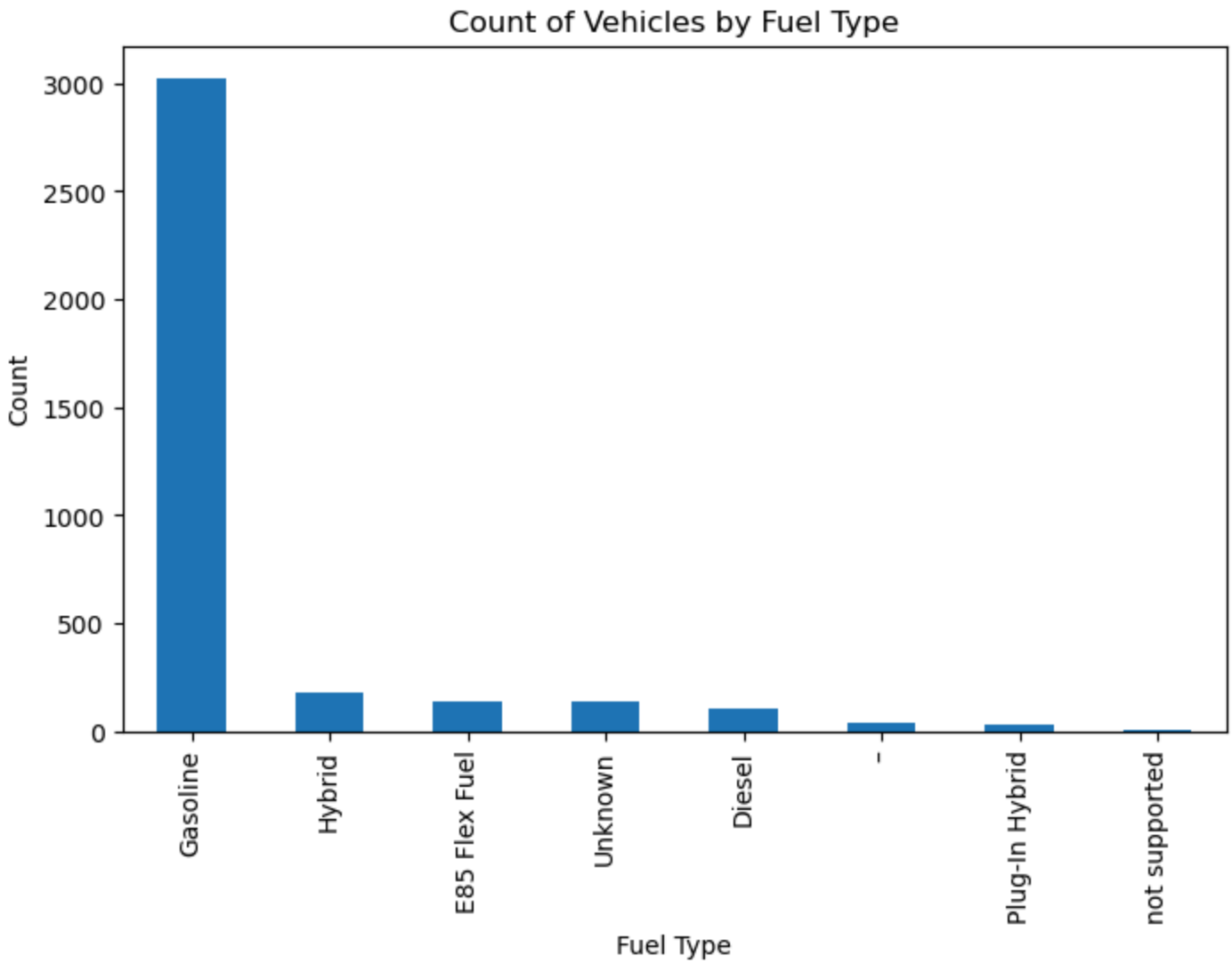
```
# Histogram: Log Price
plt.figure(figsize=(8, 5))
plt.hist(df["log_price"].dropna(), bins=30, edgecolor="black")
plt.title("Distribution of Log Price")
plt.xlabel("Log Price")
```

```
plt.ylabel("Frequency")
plt.show()
```



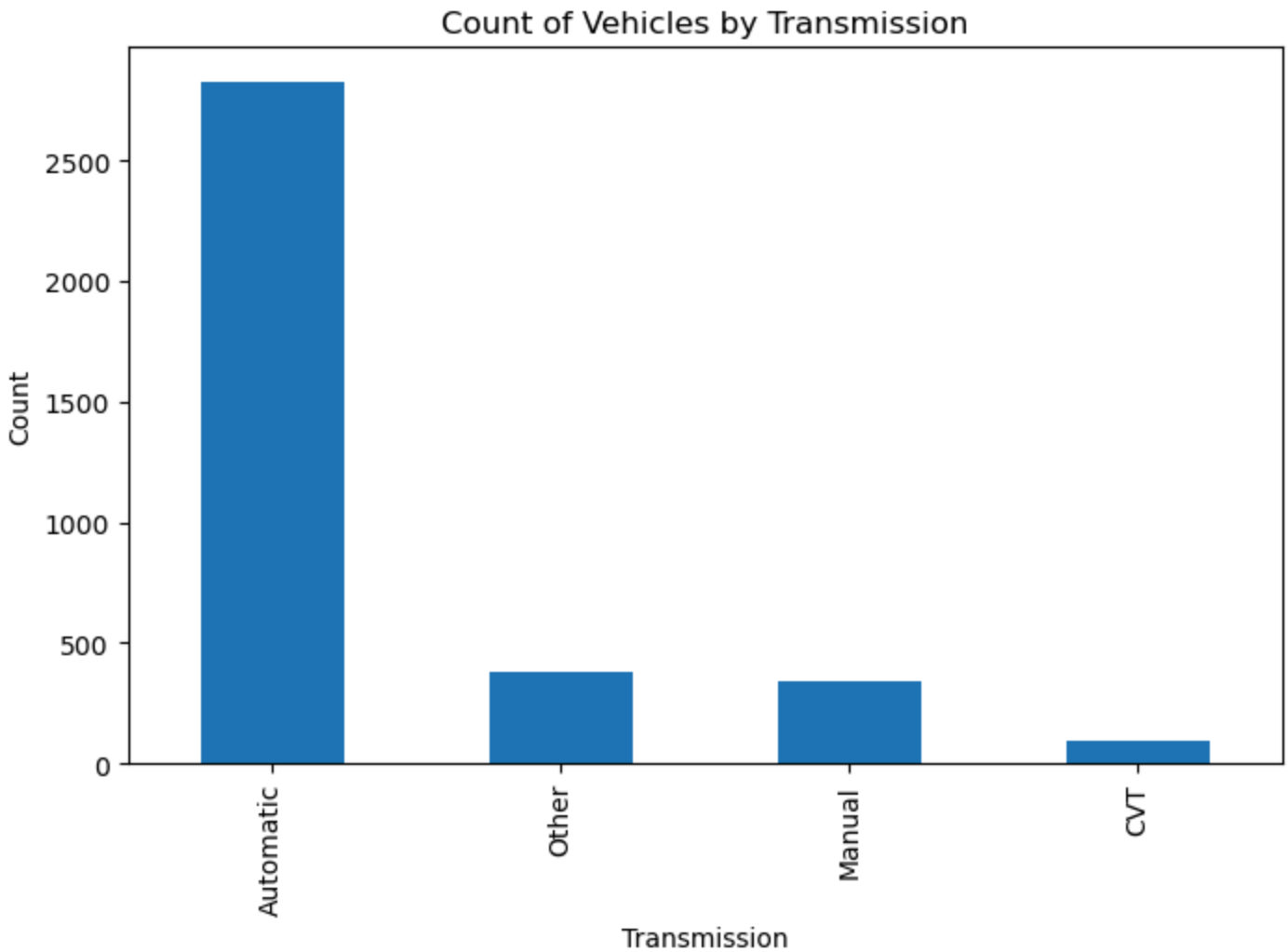
In [20]:

```
# Bar chart: Fuel Type
plt.figure(figsize=(8, 5))
df["fuel_type"].value_counts().plot(kind="bar")
plt.title("Count of Vehicles by Fuel Type")
plt.xlabel("Fuel Type")
plt.ylabel("Count")
plt.show()
```



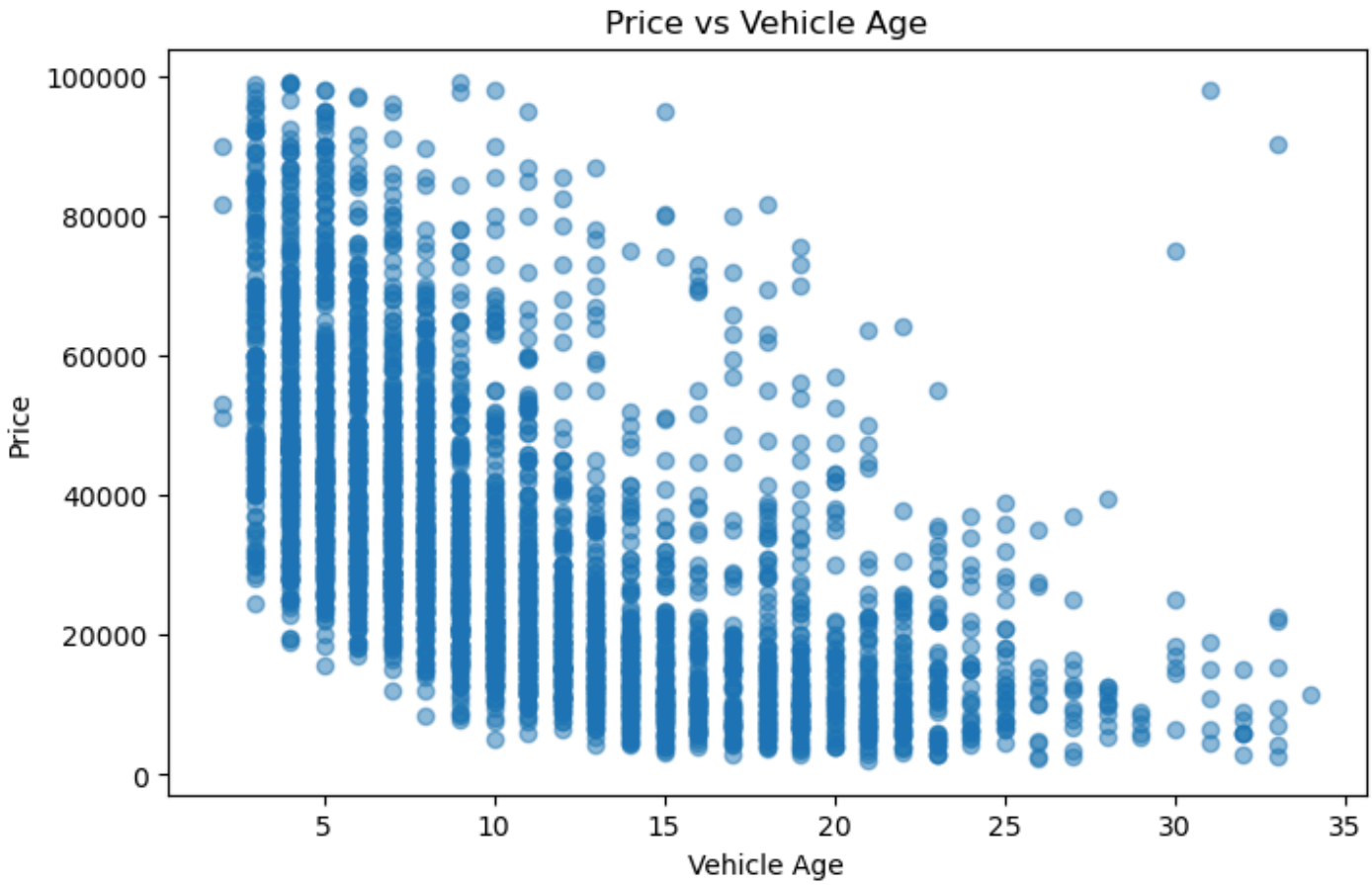
In [21]:

```
# Bar chart: Transmission
plt.figure(figsize=(8, 5))
df["transmission_group"].value_counts().plot(kind="bar")
plt.title("Count of Vehicles by Transmission")
plt.xlabel("Transmission")
plt.ylabel("Count")
plt.show()
```

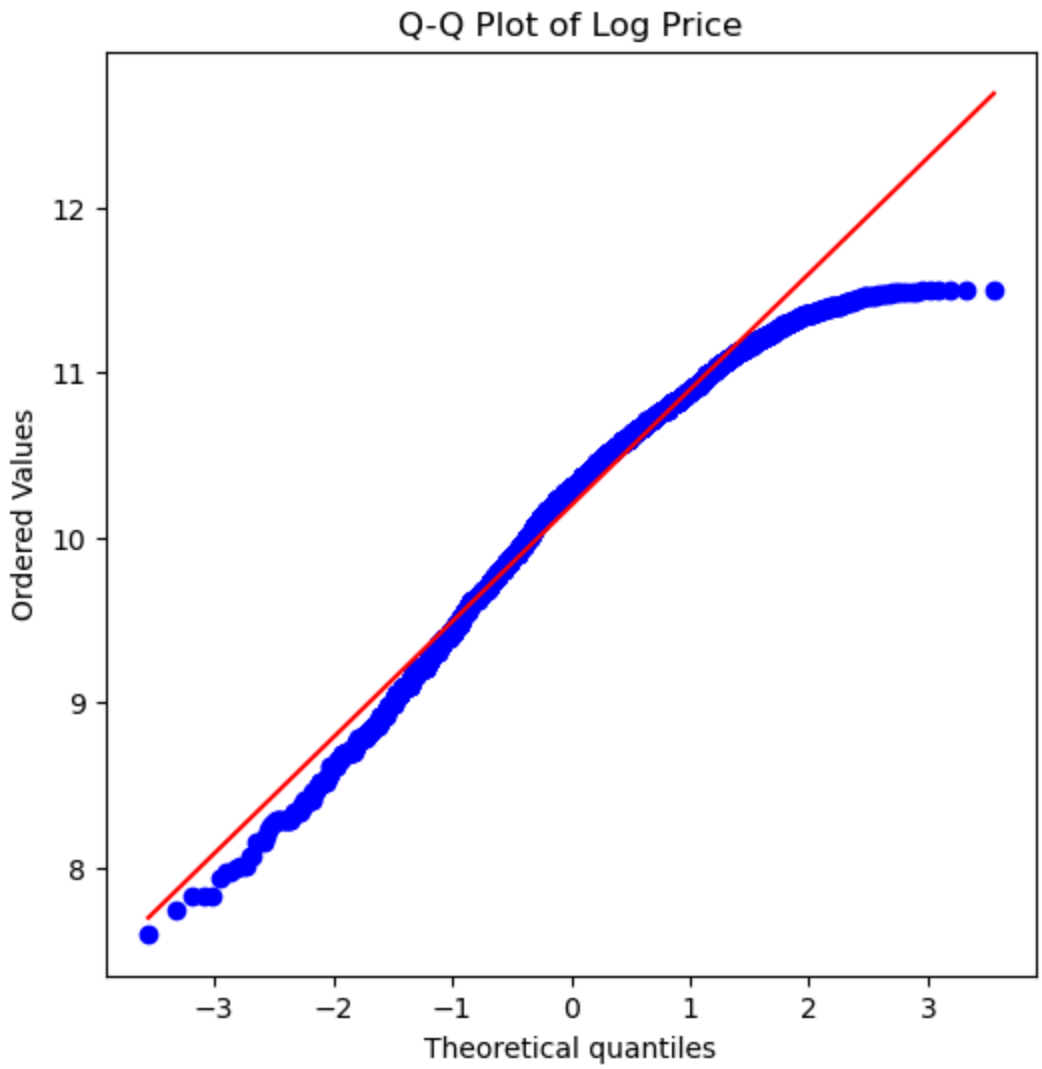


In [22]:
```
# Scatter: Price vs Mileage
plt.figure(figsize=(8, 5))
plt.scatter(df["mileage"], df["price"], alpha=0.5)
plt.title("Price vs Mileage")
plt.xlabel("Mileage")
plt.ylabel("Price")
plt.show()
```

In [23]:
```
# Scatter: Price vs Vehicle Age
plt.figure(figsize=(8, 5))
plt.scatter(df["vehicle_age"], df["price"], alpha=0.5)
plt.title("Price vs Vehicle Age")
plt.xlabel("Vehicle Age")
plt.ylabel("Price")
plt.show()
```



In [24]:
```
# Q-Q plot for log price
plt.figure(figsize=(6, 6))
probplot(df["log_price"].dropna(), dist="norm", plot=plt)
plt.title("Q-Q Plot of Log Price")
plt.show()
```



In [25]:
```
# Selecting features and target
features = [
    "mileage",
    "vehicle_age",
    "horsepower",
    "engine_liters",
    "fuel_type",
    "transmission_group",
    "brand_group",
    "accident_flag",
    "clean_title_flag"
]

target = "log_price"

X = df[features]
y = df[target]
```

In [26]:
```
# Train/test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42
)

print("\nTraining set shape:", X_train.shape)
print("Testing set shape:", X_test.shape)
```

```
Training set shape: (2918, 9)
Testing set shape: (730, 9)
```

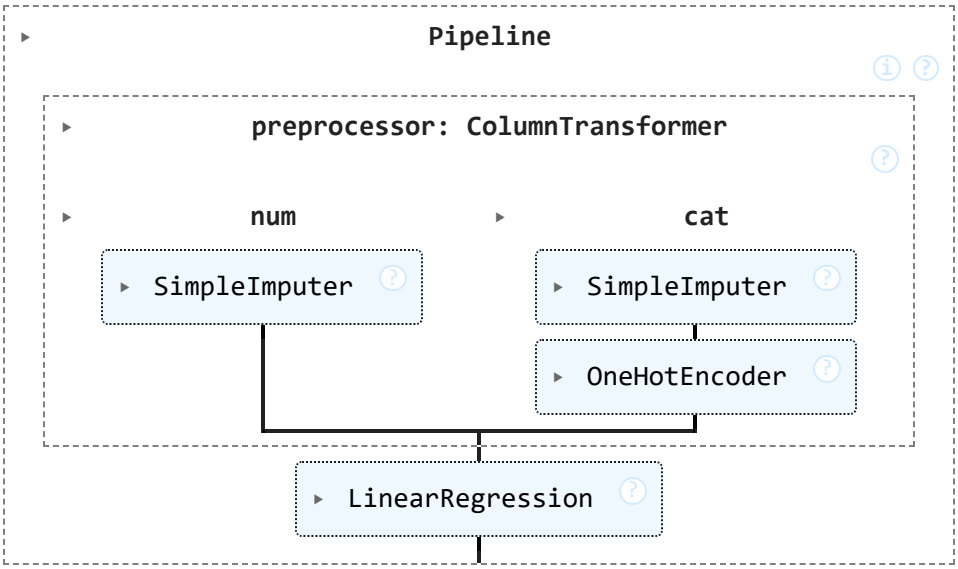
In [27]:

```python
# Preprocesssing pipeline
numeric_features = [
    "mileage",
    "vehicle_age",
    "horsepower",
    "engine_liters",
    "accident_flag",
    "clean_title_flag"
]

categorical_features = [
    "fuel_type",
    "transmission_group",
    "brand_group"
]

numeric_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="median"))
])

categorical_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore"))
])

preprocessor = ColumnTransformer(
    transformers=[
        ("num", numeric_transformer, numeric_features),
        ("cat", categorical_transformer, categorical_features)
    ]
)
```

In [28]:

```python
# Multiple linear regression model
model = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("regressor", LinearRegression())
])

model.fit(X_train, y_train)
```

Out[28]:



In [29]:

```
# Predictions
y_pred_log = model.predict(X_test)

# Convert back to original dollar scale
y_test_price = np.expm1(y_test)
y_pred_price = np.expm1(y_pred_log)
```

In [30]:

```
# Model evaluation
r2_log = r2_score(y_test, y_pred_log)
r2_price = r2_score(y_test_price, y_pred_price)
rmse = np.sqrt(mean_squared_error(y_test_price, y_pred_price))
mae = mean_absolute_error(y_test_price, y_pred_price)

print("\nModel Performance:")
print("R² on log(price):", round(r2_log, 4))
print("R² on original price scale:", round(r2_price, 4))
print("RMSE:", round(rmse, 2))
print("MAE:", round(mae, 2))

if r2_log >= 0.70:
    print("The model meets the 70% accuracy threshold based on R².")
else:
    print("The model does NOT meet the 70% accuracy threshold based on R².")
```

```
Model Performance:
R² on log(price): 0.7537
R² on original price scale: 0.6476
RMSE: 12567.41
MAE: 8282.31
The model meets the 70% accuracy threshold based on R².
```

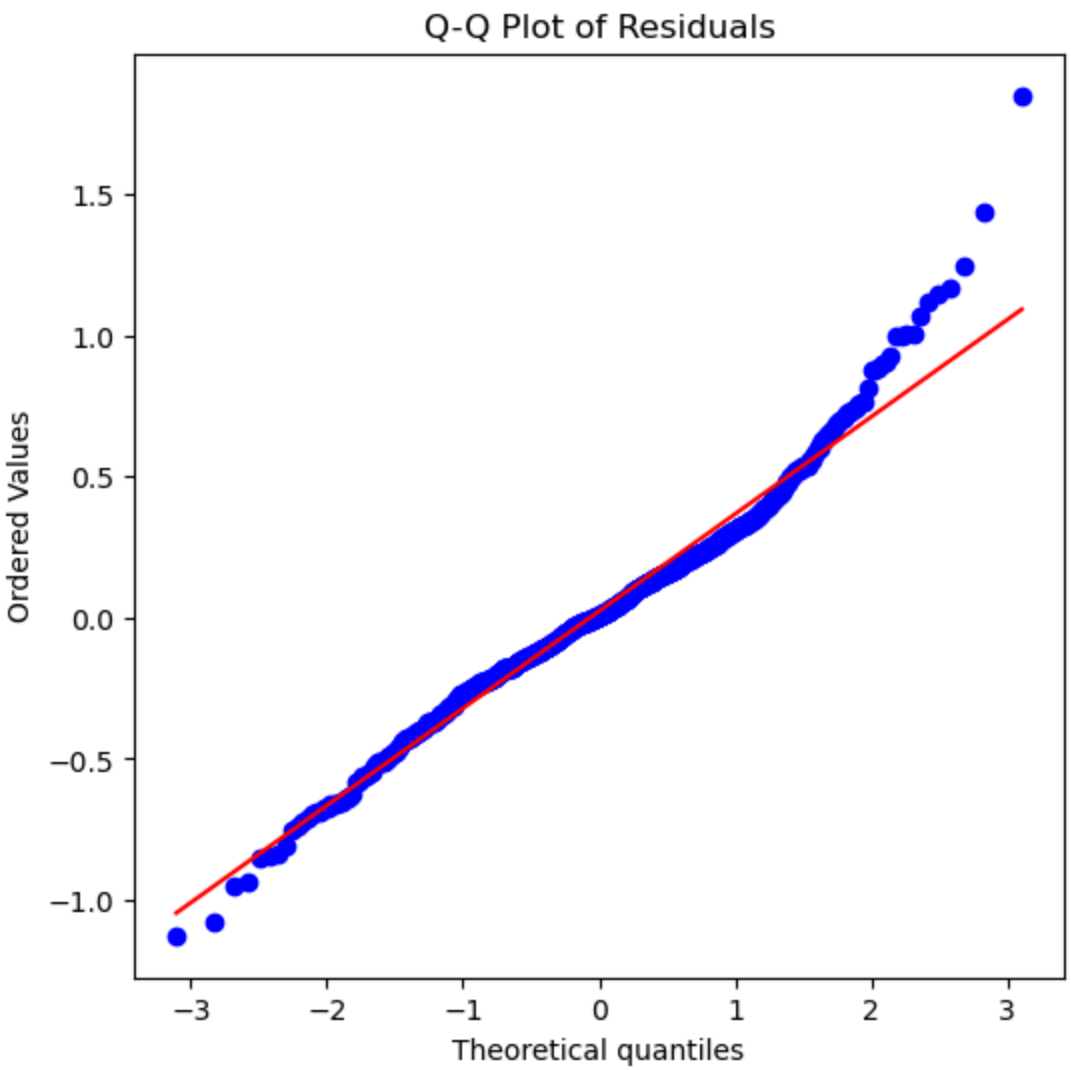
In [31]:

```
# Residual check
residuals = y_test - y_pred_log
residual_stat, residual_p = normaltest(residuals)

print("\nResidual normality test p-value:", residual_p)

residual_normality_df = pd.DataFrame({
    "Statistic": [residual_stat],
    "p_value": [residual_p]
```

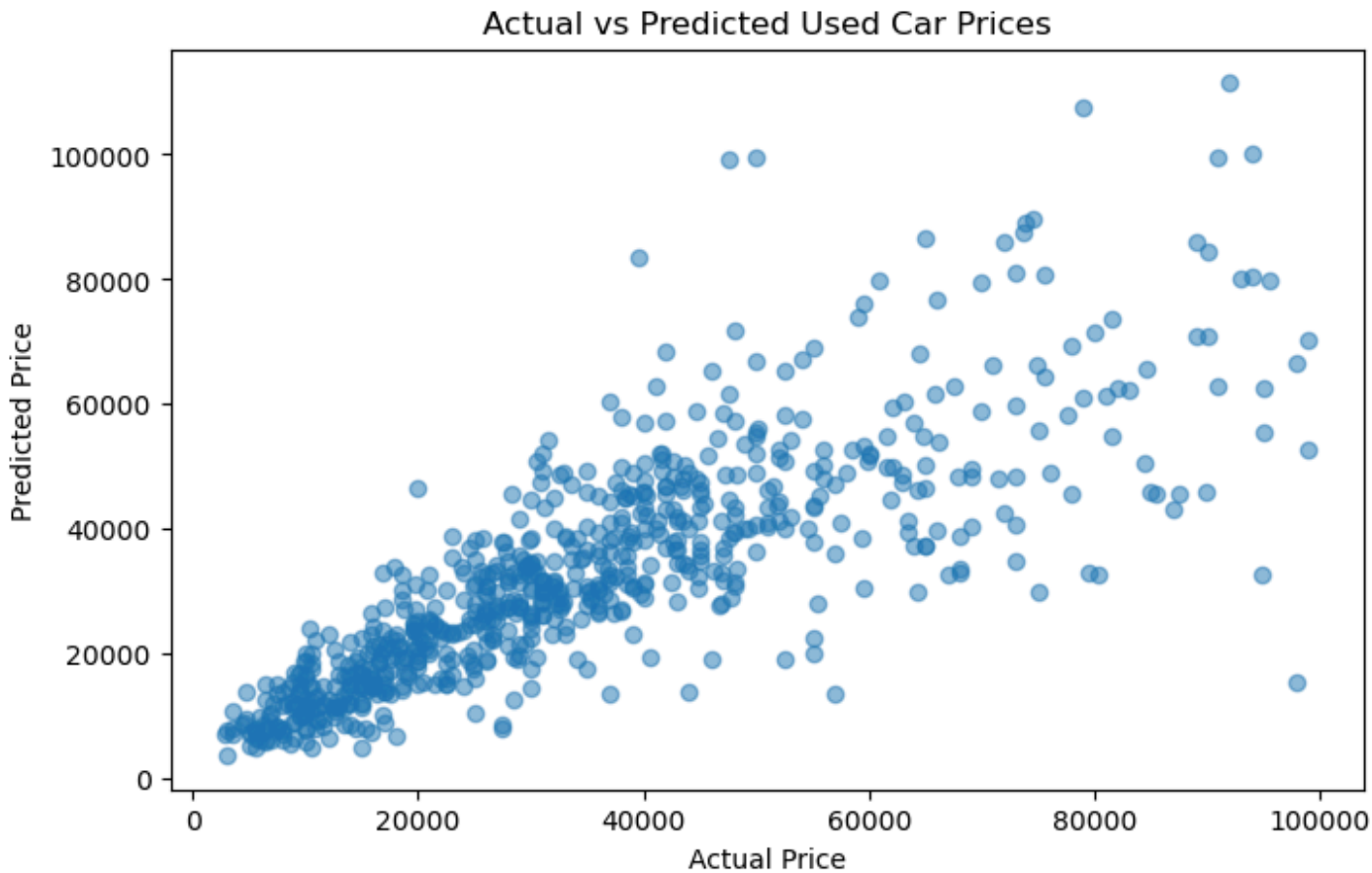
```
})
residual_normality_df.to_csv(OUTPUT_DIR / "residual_normality.csv", index=False)
```

```
Residual normality test p-value: 2.6356397943137252e-14
```

In [32]:
```
# Q-Q plot of residuals
plt.figure(figsize=(6, 6))
probplot(residuals, dist="norm", plot=plt)
plt.title("Q-Q Plot of Residuals")
plt.show()
```



In [33]:
```
# Actual vs predicted plot
plt.figure(figsize=(8, 5))
plt.scatter(y_test_price, y_pred_price, alpha=0.5)
plt.title("Actual vs Predicted Used Car Prices")
plt.xlabel("Actual Price")
plt.ylabel("Predicted Price")
plt.show()
```



In [34]:
```
# Regression coefficients
feature_names = model.named_steps["preprocessor"].get_feature_names_out()
coefficients = model.named_steps["regressor"].coef_

coef_df = pd.DataFrame({
    "Feature": feature_names,
    "Coefficient": coefficients
})

coef_df["Absolute_Coefficient"] = coef_df["Coefficient"].abs()
coef_df = coef_df.sort_values(by="Absolute_Coefficient", ascending=False)

print("\nTop 15 coefficients:")
print(coef_df.head(15))

coef_df.to_csv(OUTPUT_DIR / "regression_coefficients.csv", index=False)
```

```
Top 15 coefficients:
                              Feature  Coefficient  Absolute_Coefficient
30             cat__brand_group_Other    -0.206442              0.206442
31           cat__brand_group_Porsche     0.152928              0.152928
16     cat__transmission_group_Manual     0.094176              0.094176
14  cat__transmission_group_Automatic    -0.082254              0.082254
6               cat__fuel_type_Diesel     0.081213              0.081213
4                  num__accident_flag    -0.079725              0.079725
8             cat__fuel_type_Gasoline    -0.062769              0.062769
29            cat__brand_group_Nissan    -0.057112              0.057112
3                  num__engine_liters     0.056558              0.056558
1                    num__vehicle_age    -0.046610              0.046610
9               cat__fuel_type_Hybrid     0.045514              0.045514
33            cat__brand_group_Toyota     0.041643              0.041643
27             cat__brand_group_Lexus     0.040963              0.040963
11             cat__fuel_type_Unknown    -0.040043              0.040043
19               cat__brand_group_BMW     0.037454              0.037454
```

In [35]:

```python
# Save cleaned data and model summary
df.to_csv(OUTPUT_DIR / "cleaned_used_cars.csv", index=False)

prediction_results = pd.DataFrame({
    "Actual Price": y_test_price.values,
    "Predicted Price": y_pred_price,
    "Residual (Log Scale)": residuals.values
})
prediction_results.to_csv(OUTPUT_DIR / "prediction_results.csv", index=False)

model_summary = pd.DataFrame({
    "Metric": ["R2_log_price", "R2_price_scale", "RMSE", "MAE", "Residual_Normality_p"],
    "Value": [r2_log, r2_price, rmse, mae, residual_p]
})
model_summary.to_csv(OUTPUT_DIR / "model_summary.csv", index=False)

print("\nAll outputs saved to:")
print(OUTPUT_DIR)
```

```
All outputs saved to:
output
```

In [ ]:

In [ ]: